1.5

1. If/elif blocks: My main program uses if/elif to select product types, while the new discount strategies replace the need for discount-related if/elif blocks in Order.

2. Product creation: Previously, products were created directly in the main program and JSON loader. These now use ProductFactory.

3. Possible errors: Entering an invalid price or loading a missing file could cause errors if they are not handled.

2

Predict First 1: Adding StudentDiscount

Prediction: Without Strategy, I would need to change the calculate_total() method in Order. With Strategy, I think I can just add a StudentDiscount class.

Actual Result: I added StudentDiscount without changing Order, and the test passed.

Predict First 2: Changing Discounts

Prediction: I think BulkDiscount(200, 20) will give $230, while PercentageDiscount(10) will give $225 for a $250 order.

Actual Result: The totals were $230 and $225 respectively.

Predict First 3: Relationship

Prediction: I think Order and DiscountStrategy have an association because Order uses a discount but does not own it.

Actual Result: Two Orders could share the same discount strategy object.

Explanation: The relationship is association. Each Order uses one strategy, while a strategy can be shared by multiple Orders.

3

Predict First 4: Unknown Product Type

Prediction: I think using "service" as the product type will raise a ValueError because the Factory only supports physical and digital products.

Actual Result: The Factory raised "Unknown product type: service".

Explanation: The error comes from the final else block in ProductFactory.create(). Without exception handling, it would stop the program.

Predict First 5: Adding BundleProduct

Prediction: Before Factory, I would need to update the main program and file loader. After Factory, I would mainly need to update ProductFactory and add the new class.

Actual Result: Product creation is now handled by ProductFactory instead of being repeated in different parts of the application.

Explanation: The Factory still needs changes when adding a new product type, but it keeps the creation logic in one place.

4

Predict First 6: DigitalProduct Stock

Prediction: I think DigitalProduct should not be allowed to skip the stock validation because it inherits from Product.

Actual Result: DigitalProduct raised a ValueError when negative stock was entered.

Explanation: Both subclasses must follow the same basic rules to satisfy LSP.

5

Predict First 7: Invalid User Input

Prediction: I think "abc" will show an invalid number message, "-5" will ask for a positive number, and "12.50" will be accepted.

Actual Result: The first two inputs were rejected, and the function returned 12.5.

Explanation: The program keeps asking until a valid positive number is entered.

Predict First 8: CSV Loading

Prediction: I think 3 products will load. The unknown product type and invalid price will be skipped.

Actual Result: Desk Lamp, Photo Editor and Monitor Stand loaded. The other two rows were skipped, followed by "3 products loaded." and "Load attempt finished."

Explanation: Each invalid row is handled separately, so the remaining products can still load.

Predict First 9: Missing CSV File

Prediction: I think the except and finally blocks will run, but the else block will not.

Actual Result: The program displayed the missing-file message and then "Load attempt finished."

Explanation: The except block handles the missing file, while finally always runs.